The SVD of the Fredy dependency matrix

9135 declarations, 126 847 dependency edges · computed 2026-07-03

The matrix

A is 9135×9135 with $A_{ij} = 1$ when declaration i **uses** declaration j (the constants `#print axioms` walks), else 0. It is very sparse: 126 847 ones out of 83 million entries (0.15%). Read each **row** as one declaration's checklist of what it uses.

The eigenvalues are useless here: the dependency graph has no cycles, so A is (after reordering) triangular with a zero diagonal and **every eigenvalue is 0**. The SVD works anyway — singular values are always real and ≥ 0.

What the SVD is

The SVD writes the whole matrix as a sum of simple blocks, biggest first:

$$A = \sigma_1 \, u_1 v_1^T + \sigma_2 \, u_2 v_2^T + \sigma_3 \, u_3 v_3^T + \cdots$$

Each $u_r v_r^T$ is an outer product — a rank-1 matrix whose (i, j) entry is one number, $\sigma_r \cdot u_r[i] \cdot v_r[j]$: the block's guess for **"does declaration i use dependency j"**, a per-declaration number times a per-dependency number. So one block is **a group of declarations that share a group of dependencies**, and σ_r says how large that block is. The three pieces:

- v_r (a column of V): a weighting over the 9135 **dependencies** — a bundle of things used together.
- u_r (a column of U): a weighting over the 9135 **declarations** — who draws on that bundle.
- σ_r (a diagonal entry of S, zero off-diagonal): the size of block r.

The vectors are unit length, so u_r's entries are tiny (spread over 9000 declarations) while v_r's concentrate on a few dependencies. Signs are arbitrary (flip u_r and v_r together); v_2, v_3 come out as **contrasts** (some $+$, some $-$), which is how a block separates two groups.

The singular values, and why σ_1^2 is 14%

r	1	2	3	4	5	6	7	8	9	10	11	12
σ_r	133.2	81.8	57.7	53.3	50.9	48.3	45.4	43.2	39.5	38.9	36.3	35.3
σ_r^2/Σ (%)	13.98	5.28	2.63	2.24	2.04	1.84	1.63	1.47	1.23	1.19	1.04	0.98

The "% of what" is the point. The squared size of the matrix is

$$\|A\|_F^2 = \sum_{i,j} A_{ij}^2 = (\text{number of ones}) = 126847,$$

because the entries are 0 or 1, so $A_{ij}^2 = A_{ij}$. A basic SVD identity says this same number is the sum of the squared singular values:

$$\|A\|_F^2 = \sum_r \sigma_r^2 = 126847.$$

So each σ_r^2 is a slice of one fixed total. The first block:

$$\sigma_1^2 = 133.2^2 = 17742, \quad 17742/126847 = 13.98\% \approx 14\%.$$

That is what "14%" means: **14% of all the 1s in the matrix are accounted for by the single biggest co-usage block.** No other block is close (next is 5.3%), and it takes all top 12 to reach 35.5% — the structure is spread thin, not concentrated in a few blocks.

The top three dependency bundles v_1, v_2, v_3

v_1 — $\sigma_1 = 133.2$ (14%)
the category core

+.51	`Cat.Hom`
+.41	`Cat.comp`
+.39	`Cat`
+.21	`Cat.id`
+.20	`Cat.assoc`
+.14	`prod`
+.13	`HasBinaryProducts`
+.13	`Allegory.toCat`

All positive: everything uses the basic category operations. This block **is** the fact that the whole library rests on `Cat`.

v_2 — $\sigma_2 = 81.8$ (5.3%)
allegory vs plain category

+.41	`Allegory.toCat`
−.32	`Cat`
+.24	`Alg.le`
+.23	`Allegory.recip`
+.22	`UnionAllegory…`
+.21	`distrib…isUnion`
+.19	`Cat.Hom`
+.18	`DistribAlleg…`

A contrast: + on relational/allegory machinery, − on plain `Cat`. This axis separates Chapter-2 allegory code from category code.

v_3 — $\sigma_3 = 57.7$ (2.6%)
limits vs subobjects

+.33	`Cat`
+.30	`HasBinaryProducts`
+.29	`HasTerminal`
+.21	`Cat.Hom`
−.21	`Subobject.dom`
−.20	`Subobject.arr`
−.18	`Cat.assoc`
−.17	`Subobject`

A contrast: + on finite-limit structure (products, terminal), − on subobject machinery.

The matching declarations u_1, u_2, u_3

u_r lists the declarations that draw most on bundle v_r (entries are small — each is one of 9000).

u_1 — heaviest users of the core
- `amalgamation_is_pullback`
- `monic_epic_is_cover`
- `foldExists`
- `amalgamation_lemma`
- `pushout_monic_in_pretopos`
- `capital_iff_complemented…`
- `recursor_exists_of_bicartesian`

Large high-level theorems that pull in a lot of category machinery.

u_2 — the allegory side
- `Alg.powerRel_comp`
- `Alg.dp_thin_prefixed_context`
- `Alg.mapHasBinaryCoproducts`
- `Alg.A_is_map'`
- `Alg.ellMap_kappaMap_disjoint`
- `Alg.dp_thin_prefixed`
- `Alg.A_is_map`

Every one is `Alg.*` — confirming v_2 is the allegory-vs-category axis.

u_3 — the subobject side (−)
- `monic_epic_is_cover`
- `amalgamation_lemma`
- `amalgamation_is_pullback`
- `free_peano_property…`
- `foldExists`
- `peano_property_of_bicartesian`
- `capital_iff_complemented…`

Theorems that lean on subobjects (negative end of the limits/subobjects contrast).

What u_1 is, as numbers

u_1 is a vector of 9135 real numbers — one per declaration — scaled so they square-sum to 1 (which is why each is small). All are ≥ 0 here ($\max = 0.0377$); 2143 of the 9135 are essentially zero. The actual values, sorted:

rank	u_1	**declaration**
0	+0.0377	`amalgamation_is_pullback`
1	+0.0355	`monic_epic_is_cover`
2	+0.0354	`foldExists`
50	+0.0280	`Alg.mapDisjointBinaryCoproduct`
500	+0.0195	`prodEndo_preservesProductMonic`
2000	+0.0135	`PeanoProperty`
5000	+0.0064	`Alg.globalSup_apply`
9134	≈ 0	`powerSetMap_id`

One entry means: **add up how core-ish each dependency that declaration i uses is, then divide by σ_1** — $u_1[i] = \left(\sum_{j \text{ used by } i} v_1[j]\right)/\sigma_1$. Worked for the top one: `amalgamation_is_pullback` uses 144 things; summing v_1 over exactly those $(0.508 + 0.414 + 0.394 + ...)$ gives 5.017, and $5.017/133.2 = 0.0377$ — its u_1 entry exactly. So $u_1[i]$ is "how much of the category core declaration i pulls in."

And the outer-product number: for $i =$ `amalgamation_is_pullback` ($u_1 = 0.0377$) and $j =$ `Cat.Hom` ($v_1 = 0.508$), block 1 gives $133.2 \times 0.0377 \times 0.508 = 2.55$ (actual $A_{ij} = 1$; block 1 overshoots, later blocks correct it). A light user (rank 5000, $u_1 = 0.0064$) against `Cat.Hom` gives only 0.44 — small u_1, so the block predicts "barely uses the core."

What this buys us

The top block just rediscovers the core (the same hubs plain in-degree finds). The **later** blocks are more interesting: they split the library by subject — allegory vs category (v_2), limits vs subobjects (v_3) — which is a purely mechanical read of the dependency structure, no labels used.

The concrete payoff is for duplicate detection. Give each declaration its coordinates along $u_1, ..., u_{12}$ (its row of U scaled by the σ's): a 12-number fingerprint of what it depends on. Two copies of the same lemma have the same checklist, hence the same fingerprint — the three known duplicates score cosine $0.94, 1.00, 1.00$ in this 12-D space. So the SVD gives a fast way to **propose** duplicate pairs (nearest fingerprints); the final decision still compares the two rows directly (`scripts/dep_dup.py`). The same coordinates would also give the graph viewer a deterministic layout.